交通信号灯控制 1

单行车道与人行道的红绿灯系统设计实验指导

一、实验背景与题目

随着经济建设的发展，交通日益繁忙，事故时有发生。为了保障行人和行车的安全，在车流繁忙的双向车道的人行斑马线处都设有双级人行红绿灯，以更好地控制车流。在深圳理工大学的校园外就有类似的设计，如图 1 所示。



图 1. 深圳理工大学校园外某处的双级人行红绿灯

在图中的场景下，为了简化问题并拓展场景，我们只考虑图片中行人从远景的车道侧走向双级红绿灯的中间过渡区域，并且人行道旁的红绿灯有**预备通行按钮**，按下后会根据情况**缩短人行道红灯的时间**，以避免行人等待时间过长。简化的俯视图如图 2 所示。

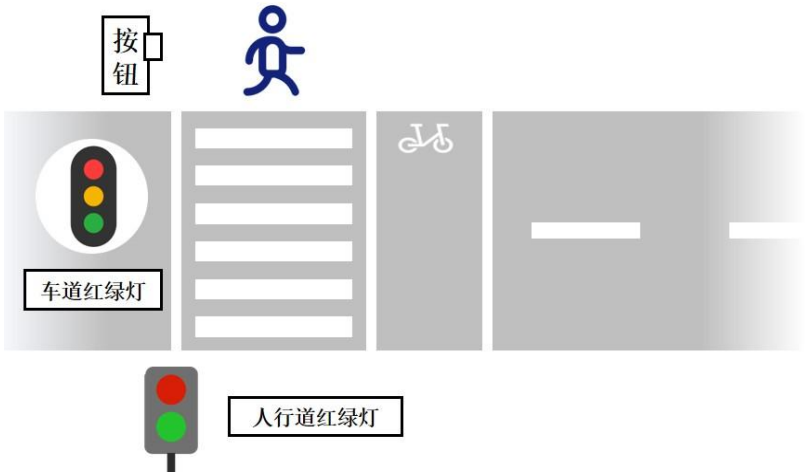


图 2. 行人从远景的车道侧走向双级红绿灯的中间过渡区域的俯视图示意

根据图 2 的模式，总结任务要求如下：

1. **车道红绿灯**有 3 个状态：红灯停、绿灯行、黄灯等待；**人行道红绿灯**则只有 2 个状态：红灯等待，绿灯通行。当车道红绿灯在绿灯和黄灯期间，**人行道红绿灯**处在红灯状态；而车道为红灯时，人行道为绿灯。

2. 通常情况下，**车道红绿灯**的绿灯维持 90T，随后是 5T 的黄灯，最后变为红灯维持 50T，再转变为绿灯，如此往复循环。每个灯的状态以 T 为单位倒数计时，**至 1T 后直接进入下一状态** (这里注意不是 0T)。**其中，T 为 10 个时钟周期**。

3. 当行人按下按钮，且**人行道红绿灯**的红灯倒计时还没有到达 10T，则车道红绿灯立即**随时钟同步**到 10T 处继续倒计时，以提早**人行道红绿灯**转变为绿色；当处在其他状态时，行人按下按钮不会对系统造成任何变化。

4. 异步复位时，车道红绿灯转变为绿灯并重新从 90T 开始重新计时。

二、实验任务

按下列要求完成实验任务，并撰写实验报告：

实验任务 1. 绘制 FSM (有限状态机)

根据背景中的要求，绘制上述系统的 FSM，**使用 Office 软件(Word 等) 绘制后保存或截图，并体现在实验报告中**。

实验任务 2. 设计电路

根据你在“实验任务 1”中的 FSM，用课上所学的“组合逻辑电路+时序逻辑电路”设计本系统的电路，并绘制该电路的状态转化表。注意关注“设计说明和提示”中“1. 有关时间 T 的说明”内对 T 的说明。**该部分需要使用 Logisim 将电路进行绘制 (不要求亮灯测试)，完成后截图体现在实验报告中**。

实验任务 3. Verilog 描述与仿真

按照“设计说明和提示”的“2. '实验任务 3'的顶层电路说明”中的要求，使用 Verilog 描述你**在“实验任务 2”中设计的电路**；同时按照下文“设计说明和提示”中“3. 波形截取操作”，完成系统的仿真和关键波形的截取。**该部分需要包含设计代码 (全部的文本内容以及包含个人信息的部分截图) 与截取的波形，要求包含从图 6 至图 9 中示意的波形与对应描述以及个人信息。**

实验报告提交要求与截止日期

提交 PDF 或 Word 文档，文件命名为”exp5_<name>_SUAT<id>.pdf/.docx”，提交给对应课程助教的企业微信或深理工邮箱 (按你在前 4 次实验课的提交助教为准)。

截止日期为两周后的周一 (即 2026 年 11 月 9 日 0:00 前)，迟交每晚 1 天扣本实验总分的 10%。

【附】设计说明和提示

1. 有关时间 T 的说明

在“实验任务 1 & 2”中，为了让大家更专注于电路本身的设计，**在该部分将 T 只定义为 1 个时钟周期**（而不是在背景中描述的 10 个时钟周期）；**但在“实验任务 3”中，Verilog 设计还是要求将 T 视作 10 个时钟周期。**

2. "实验任务 3"的顶层电路说明

为了让同学们能直接使用工程中提供的 testbench，这里提供**统一的顶层电路的模块名称与信号名称**如图 3 所示。其中，`clk` 为系统时钟，`rst_n` 为**异步复位信号**，其余信号定义如下：

- `pass_request`: 人行道红绿灯的**预备通行按钮通行信号**，信号为 1 时有效；
- `clock[7:0]`: 车道红绿灯的倒数计时器输出，用于在 testbench 中监测倒计时信息；
- `motor_xxx`: 车道红绿灯的控制信号，包括 `red`，`yellow`，`green` 三种，位宽为 1bit，信号为 1 时对应的灯变亮，为 0 时则关灭；
- `pass_xxx`: 人行道红绿灯的控制信号，包括 `red`，`green` 两种，位宽为 1bit，信号为 1 时对应的灯变亮，为 0 时则关灭；

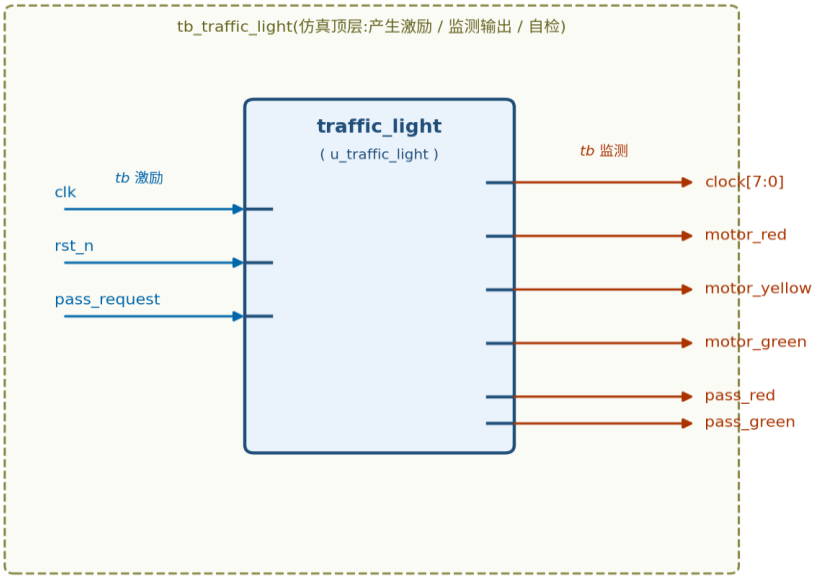


图 3. 顶层电路的模块名称与信号名称

3. 波形截取操作

本次实验的 testbench 不需要大家自己写，但本次实验中有部分输出是通过 ASCII 码显示灯的状态，而 Vivado 默认使用 16 进制进行编码且默认运行时间为 1us，远小于整体仿真时间，故而需要大家进行下述操作：

1. 运行 `Run Simulation` 后按图 4 所示操作进行完整的运行；运行完后，选中上面的 `Untitle`，鼠标移动到波形侧右击选择 `Full View`：

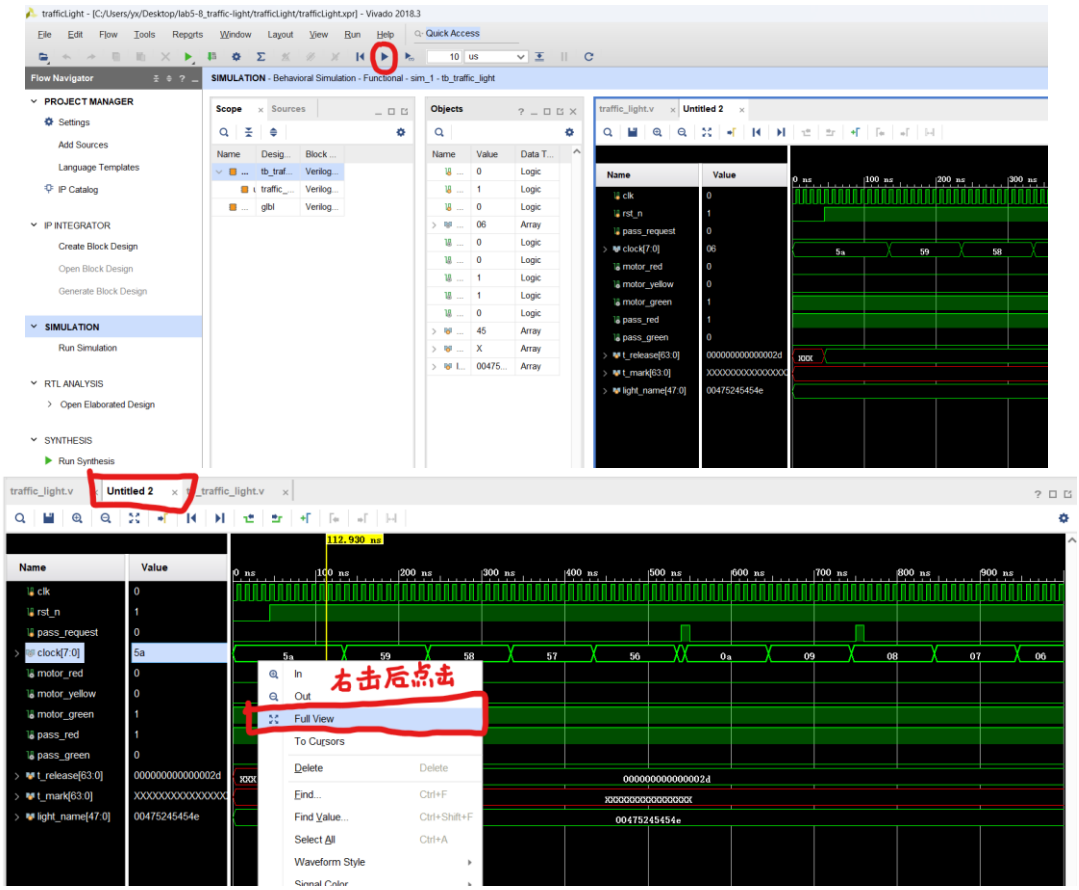


图 4. 完整运行操作并放大仿真窗口

2. 按照图 5 的操作，对 clock[7:0]和 light_name[47:0]这 2 个信号，右击对应波形后，鼠标滑动至 Radix，然后将进制转换为对应模式——clock[7:0]设为 Unsigned Decimal，light_name[47:0]设为 ASCII:

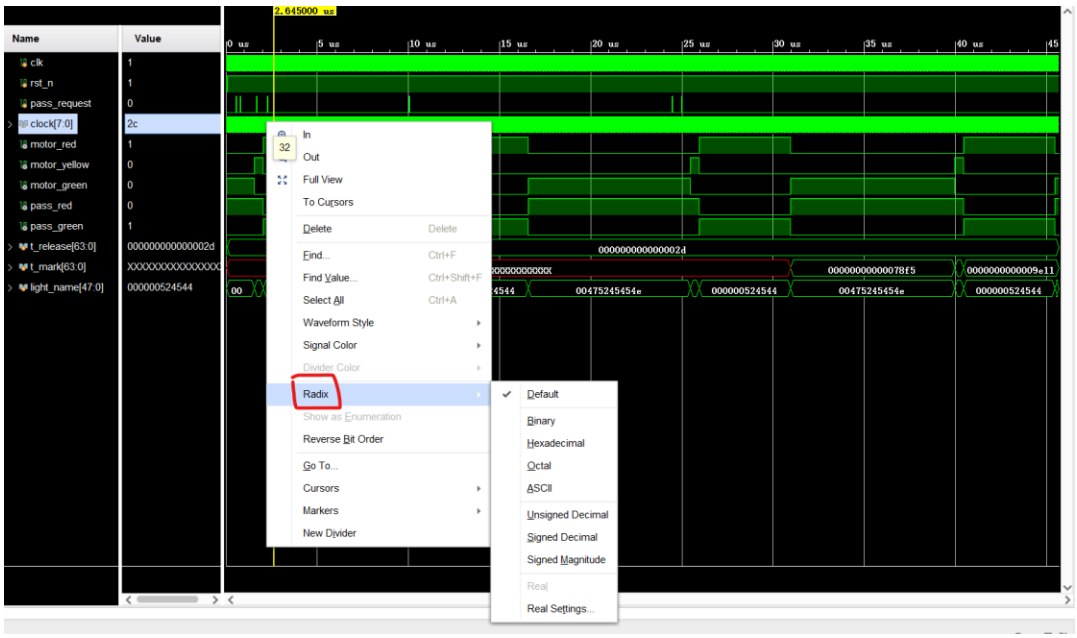


图 5. 数据格式显示转化

由于波形图整体很密集，难以直接通过截图判断大家的完成情况，故本实验需要对关键部分进行放大和截取。截取分为以下几个部分：

1. 没有行人的完整周期：找到波形中没有任何 pass_request 作用的波形部分截图，样例如下：

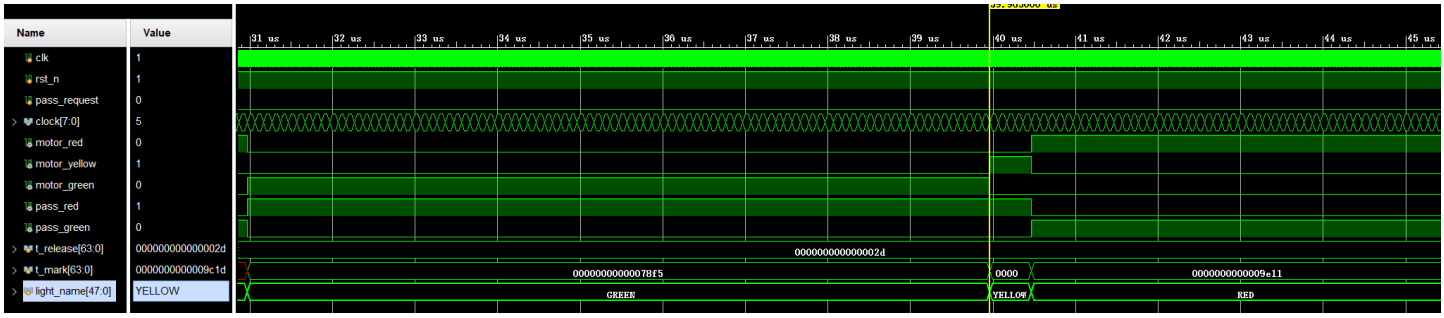


图 6. 没有行人的完整周期

2. 绿灯计时大于 10T 的部分：此时有 pass_request 作用，且绿灯倒数计时的值大于 10，对应计时器在时钟上升沿变化为 10T。找到该部分并截图，样例如下：

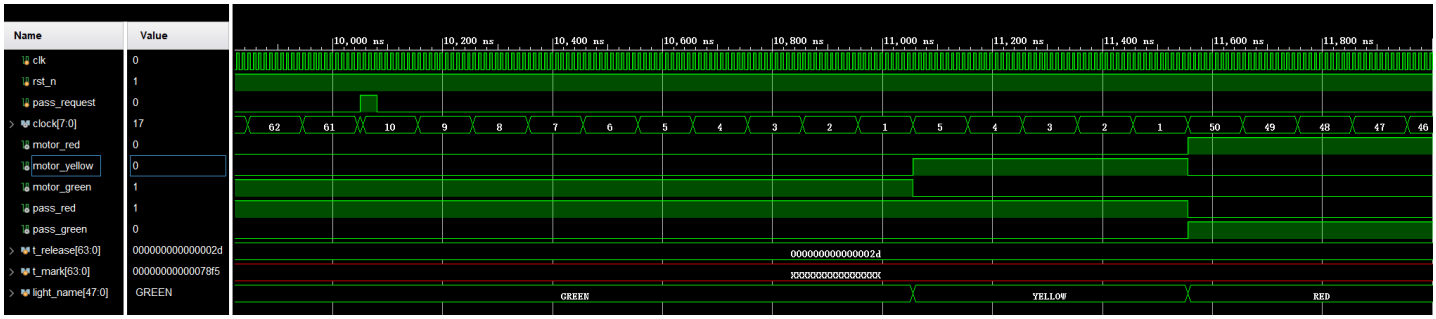


图 7. 绿灯计时大于 10T 的部分

3. 绿灯计时小于 10T 的部分：此时有 pass_request 作用，但对应计时器不变化。找到该部分并截图，样例如下：

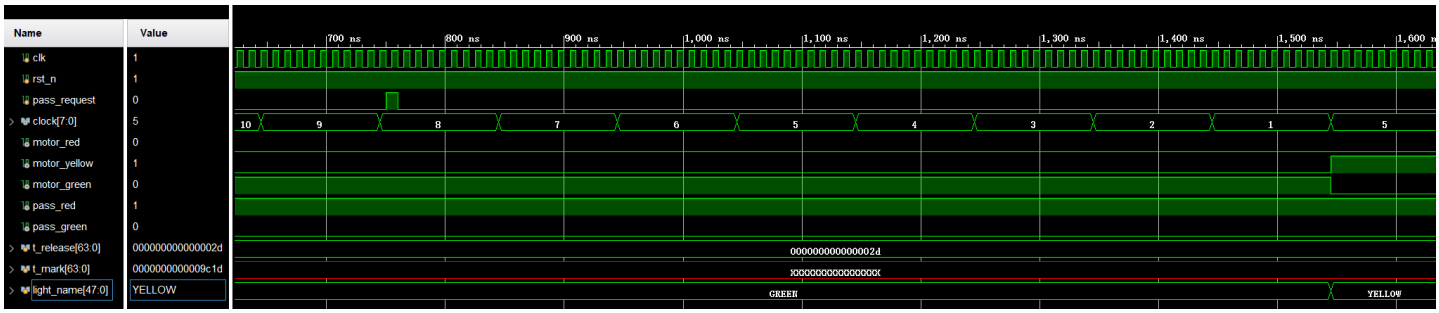


图 8. 绿灯计时小于 10T 的部分

4. 黄灯和绿灯时有行人按下按键：此时有 pass_request 作用，但对应计时器不变化。找到该部分并截图，样例如下：

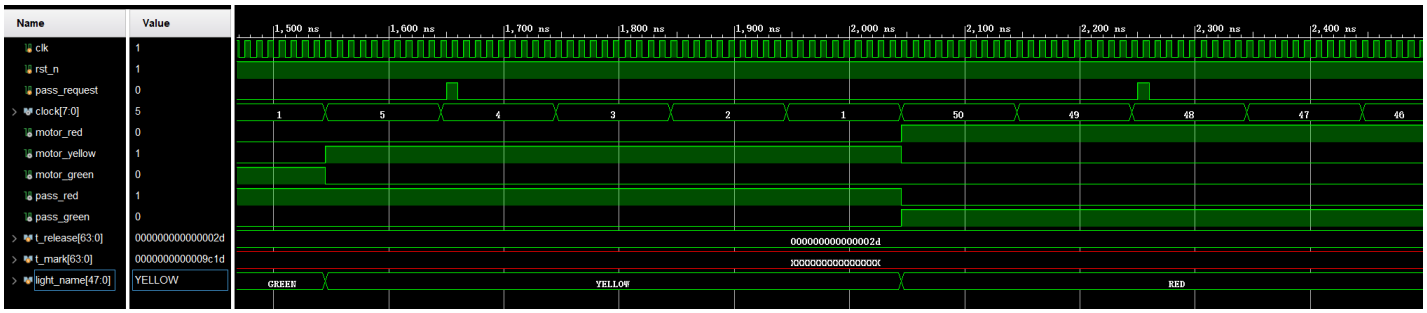


图 9. 黄灯和绿灯时有行人按下按键